

JAVA INTERVIEW PREPARATION

CHAPTER 65

LINKED LISTS AND
POINTER INVARIANTS

Senior / Lead Java Developer Textbook Edition

Full reading material - not summarized

Linked Lists: Pointer Invariants, Ownership, and Tradeoffs

Senior / Lead Java Developer Reading Chapter

Linked-list problems test whether a candidate can reason about identity and mutation without losing references. The algorithms are usually short; the difficulty lies in preserving the unprocessed chain, reconnecting boundaries, and stating which nodes may be reused.

In production Java, linked lists are less common than interview questions suggest. Arrays and array-backed collections often have better locality and lower per-element overhead. Linked structures remain useful when nodes already exist independently, insertion or removal occurs through known node references, or intrusive ordering is part of the model.

```
head
|
v
[A | next] -> [B | next] -> [C | null]
```

Singly, doubly, and circular lists

A singly linked node points to the next node. A doubly linked node also points backward. A circular list connects the tail to the head or a sentinel rather than `null`.

```
final class Node<T> {
    T value;
    Node<T> next;

    Node(T value) {
        this.value = value;
    }
}
```

Given only a head, indexing the i th node is $O(i)$. Insertion at the head is $O(1)$. Appending is $O(1)$ only when a tail reference is maintained; otherwise finding the tail is $O(n)$. Removing after a known predecessor is $O(1)$, but finding that predecessor is usually $O(n)$.

A doubly linked list spends another reference per node to support backward traversal and $O(1)$ removal of a known node. Every mutation must maintain two directions.

The real cost in Java

Each ordinary node is an object with object-header, alignment, value reference, and link references. Nodes are allocated separately and may be scattered through the heap. Traversal performs pointer chasing and tends to use CPU caches less effectively than contiguous arrays.

Big-O says both `ArrayList` iteration and linked-list iteration are $O(n)$, but constants differ substantially. `ArrayList` also provides $O(1)$ random access, while linked indexing is linear. Choose a linked structure for its ownership and mutation properties, not because insertion is generically "faster."

Garbage collection automatically reclaims unreachable nodes; it does not prevent logical leaks. A retained head, tail, cache entry, listener, or iterator can keep the entire remaining chain reachable.

Sentinels simplify boundaries

A dummy or sentinel node represents a stable position before the first real element. It avoids special handling when the head changes.

```
dummy -> A -> B -> C
      ^
stable predecessor even if A is removed
```

```
static Node<Integer> removeValue(Node<Integer> head, int target) {
    Node<Integer> dummy = new Node<>(0);
    dummy.next = head;
    Node<Integer> current = dummy;
    while (current.next != null) {
        if (current.next.value == target) {
            current.next = current.next.next;
        } else {
            current = current.next;
        }
    }
    return dummy.next;
}
```

The invariant is that all nodes before `current.next` have been processed and no retained node before that boundary matches the target.

Sentinels are especially useful in merging, partitioning, and doubly linked cache structures. They are implementation nodes and should not escape as business data.

Iterative reversal

Reversal requires preserving the unprocessed suffix before changing `next`.

```
static <T> Node<T> reverse(Node<T> head) {
    Node<T> previous = null;
    Node<T> current = head;
    while (current != null) {
        Node<T> next = current.next; // preserve suffix
        current.next = previous;      // reverse one edge
        previous = current;
        current = next;
    }
    return previous;
}
```

reversed prefix	unprocessed suffix
null <- A <- B	C -> D -> null
^prev	^current

The invariant is that `previous` heads a fully reversed prefix, while `current` heads the untouched suffix. On termination the suffix is empty and `previous` is the new head.

Time is $O(n)$ and extra space is $O(1)$. Recursive reversal is elegant but uses $O(n)$ stack frames and risks `StackOverflowError` on unbounded input.

Reverse a subrange

To reverse positions `[left, right]`, use a dummy node to locate the predecessor, reverse exactly the range, then reconnect both boundaries.

```
before -> first -> ... -> last -> after
before -> last -> ... -> first -> after
```

The four references matter: `before`, `first`, `last`, and `after`. Many incorrect implementations reverse the internal edges but lose the suffix or fail when `left` is the head.

State whether positions are zero- or one-based and validate bounds. In production code, a specialized mutable-list operation may be clearer than exposing pointer manipulation broadly.

Fast and slow pointers

When one pointer moves twice as fast as another, their relative movement reveals structure without extra memory.

To find a middle node:

```
static <T> Node<T> middle(Node<T> head) {
    Node<T> slow = head;
    Node<T> fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
    }
    return slow;
}
```

For even length this returns the second middle. Changing initialization or loop conditions can return the first. Define the contract rather than treating either as universally correct.

This pattern also splits a list for merge sort and detects cycles.

Floyd cycle detection

In a cyclic list, a fast pointer eventually laps a slow pointer. In an acyclic list, the fast pointer reaches `null`.

```
static <T> boolean hasCycle(Node<T> head) {
    Node<T> slow = head;
    Node<T> fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
        if (slow == fast) return true;
    }
    return false;
}
```

Compare node identity with `==`, not values. Two different nodes can contain equal values.

After a meeting, reset one pointer to the head and move both one step at a time; their next meeting is the cycle entry. The proof follows distances: if the noncyclic prefix has length `mu` and the cycle has length `lambda`, the meeting offset makes the remaining distance align modulo `lambda`.

Find cycle entry

```

static <T> Node<T> cycleEntry(Node<T> head) {
    Node<T> slow = head;
    Node<T> fast = head;
    do {
        if (fast == null || fast.next == null) return null;
        slow = slow.next;
        fast = fast.next.next;
    } while (slow != fast);

    slow = head;
    while (slow != fast) {
        slow = slow.next;
        fast = fast.next;
    }
    return slow;
}

```

The **do** loop avoids falsely accepting the initial equality at the head. Always guard **fast** before advancing twice.

Cycle detection is useful beyond explicit lists: repeated-state processes, functional graphs, and ownership structures can have the same shape, though equality and transition cost must be defined.

Remove the nth node from the end

Create a gap of **n** nodes between two pointers. When the leading pointer reaches the end, the trailing pointer is immediately before the node to remove.

```

static <T> Node<T> removeFromEnd(Node<T> head, int n) {
    if (n <= 0) throw new IllegalArgumentException();
    Node<T> dummy = new Node<>(null);
    dummy.next = head;
    Node<T> lead = dummy;
    Node<T> follow = dummy;

    for (int i = 0; i < n; i++) {
        lead = lead.next;
        if (lead == null) throw new IllegalArgumentException("n exceeds length");
    }
    while (lead.next != null) {
        lead = lead.next;
        follow = follow.next;
    }
    follow.next = follow.next.next;
    return dummy.next;
}

```

The sentinel makes removal of the head identical to any other removal. The gap invariant avoids a preliminary length pass.

Merge two sorted lists

Merging reuses the smallest current head and advances only that list.

```

static Node<Integer> mergeSorted(Node<Integer> a, Node<Integer> b) {
    Node<Integer> dummy = new Node<>(0);
    Node<Integer> tail = dummy;
    while (a != null && b != null) {
        if (a.value <= b.value) {
            tail.next = a;
            a = a.next;
        } else {
            tail.next = b;
            b = b.next;
        }
        tail = tail.next;
    }
    tail.next = a != null ? a : b;
    return dummy.next;
}

```

Time is $O(m + n)$ and pointer space is constant. This destructive version reuses nodes and changes ownership of both inputs. A nonmutating contract must allocate copies.

The choice on equality ($\leq$) makes the merge stable with nodes from *a* before equal nodes from *b*.

Merge sort for linked lists

Linked lists support merge sort naturally: find the middle, split, recursively sort each half, and merge. Time is $O(n \log n)$ with $O(\log n)$ recursion stack for balanced splits.

Unlike array merge sort, merging can relink nodes without an $O(n)$ auxiliary array. However, pointer chasing and per-node overhead still affect real performance. Java library list sorting may copy elements to arrays; do not infer implementation from abstract complexity.

Break the link at the split or recursion will continue over the original full chain. Also ensure cycle-free input before algorithms that expect termination.

Intersection by identity

Two acyclic singly linked lists intersect only if they eventually share the same node objects and therefore the same suffix.

The pointer-switching method traverses equal total distance:

```

static <T> Node<T> intersection(Node<T> a, Node<T> b) {
    Node<T> p = a;
    Node<T> q = b;
    while (p != q) {
        p = p == null ? b : p.next;
        q = q == null ? a : q.next;
    }
    return p;
}

```

Each pointer walks $\text{lengthA} + \text{lengthB}$. They meet at the shared node or both become *null*. Value equality is irrelevant.

Cycles complicate intersection and require classifying cycle entries first. State input assumptions in an interview rather than silently producing a nonterminating algorithm.

Palindrome checking

An $O(1)$ -space approach finds the middle, reverses the second half, compares halves, and optionally restores the list.

Restoration matters when the caller retains ownership. An interview solution that leaves a read-only input mutated violates an implicit contract even if it returns the correct boolean.

Alternative stack-based checking uses $O(n)$ space but avoids mutation and may be easier to reason about. Senior judgment includes choosing clarity when constraints do not require in-place transformation.

Doubly linked invariants

For a node `x` inside a doubly linked list:

```
x.prev.next == x
x.next.prev == x
```

Removal reconnects neighbors before optionally clearing removed links:

```
static <T> void unlink(DNode<T> x) {
    x.prev.next = x.next;
    x.next.prev = x.prev;
    x.prev = null;
    x.next = null;
}
```

Sentinel head and tail nodes eliminate null boundary cases. This structure appears in LRU caches: a hash map provides $O(1)$ key lookup and a doubly linked list provides $O(1)$ recency movement and eviction.

Concurrency makes these multi-step invariants non-atomic. Protect the map and list as one state transition; using a concurrent map alone does not make the combined cache correct.

LRU cache as a composite structure

```
map: key -> node
MRU sentinel <-> node <-> node <-> LRU sentinel
```

On `get`, find through the map and move the node to the most-recent end. On `put`, update or insert; if capacity is exceeded, remove the least-recent node and its map entry.

Capacity can be entry count or weight. Production caches also need TTL, loader coalescing, metrics, and concurrency. A handwritten interview cache demonstrates structure composition, not a recommendation to replace a mature cache library.

Copy a list with random references

A node may have `next` plus `random` pointing anywhere. A map from original identity to copy gives a clear $O(n)$ time and space solution: create every copied node, then wire edges.

An in-place interleaving technique inserts each copy after its original, uses adjacency to set random links, then separates the lists. It reduces auxiliary map space but temporarily mutates input and is harder to make exception-safe or concurrent.

Prefer the map unless the constraint explicitly values auxiliary space and mutation is allowed. Explain identity: node values cannot uniquely map copies when duplicates exist.

Immutable and persistent lists

An immutable singly linked list can share tails safely:

```
list1: A -> B -> C
list2: X -----^ shares B -> C
```

Prepending is constant time and existing versions remain valid. Random access remains linear and long chains can make recursion unsafe. Persistent structures work well for functional transformations and snapshots when their access pattern matches the design.

Immutability requires that node value objects also follow the intended ownership model. An immutable link structure containing mutable values is only shallowly immutable.

Concurrency and publication

Publishing a mutable linked list without synchronization can expose stale or partially connected state. Build immutable chains fully before safe publication, or protect mutations with a lock/ownership discipline.

Lock-free linked queues use atomic references and carefully designed memory-order guarantees. They are not a simple replacement for a few `synchronized` blocks. Use JDK concurrent collections unless implementing an algorithm is the task.

Iterators over a mutating custom list need a contract: fail-fast best effort, snapshot, weak consistency, or external synchronization. Do not promise thread safety because individual reference writes are atomic; list invariants span several writes.

Testing pointer algorithms

Draw nodes and assign identities. Test:

- null and one node;
- two nodes;
- head or tail removal;
- odd and even lengths;
- duplicate values in different nodes;
- cycle at head, middle, self-cycle, and no cycle;
- intersecting and disjoint lists;
- mutation permitted versus restoration required.

Useful properties include:

- reversing twice restores the same node order and identities;
- merged output contains every input node exactly once and is sorted;
- removing one node reduces reachable count by one;
- doubly linked forward/backward traversals agree;
- cycle entry is reachable and lies on the cycle.

Use identity-based visited sets in test oracles when values may repeat.

Production connections

Linked structures appear in cache recency, scheduler lists, free lists, intrusive queues, undo histories, persistent immutable sequences, and ordered registries. Most application lists should still use `ArrayList` unless measurement and mutation patterns justify nodes.

Production designs need ownership, bounds, concurrency, cancellation, and observability. A queue built as a linked list is not safe merely because enqueue is $O(1)$; unbounded growth remains a failure mode.

Common senior-level traps

"Linked-list insertion is $O(1)$." Only when the insertion position or predecessor is already known.

"Use `equals` to detect a cycle or intersection." Structural algorithms require node identity.

"Recursive reversal uses $O(1)$ space." It consumes linear call-stack space.

"Garbage collection prevents list leaks." A retained head or node keeps the reachable chain alive.

"ConcurrentHashMap makes an LRU cache concurrent." Map and linked-list mutations form one invariant.

"In-place is always better." It mutates caller-owned structure and complicates failure and concurrency.

A strong interview answer

I begin by defining node identity, list type, cycle assumptions, mutation ownership, and whether input must be restored. I use a sentinel when the head may change. For pointer mutation I preserve the unprocessed suffix before rewiring and state the invariant: reversed prefix, untouched suffix, or processed output chain. Fast/slow pointers find middles and cycles; a fixed pointer gap handles positions from the end; merge reuses sorted heads; and intersection compares node identity. I explain that linked structures have $O(1)$ local relinking but linear search, object overhead, and poor locality. For production I protect composite invariants, bound queues or caches, and normally prefer standard collections.

Chapter summary

Linked-list correctness is reference ownership made visible. Sentinels stabilize boundaries; loop invariants prevent lost chains; identity distinguishes structure from value. The deeper senior-level lesson is to choose a node structure only when its mutation or sharing properties justify its runtime costs.

Revision Notes

- Singly linked nodes provide forward traversal; doubly linked nodes maintain two-way invariants.
- Index lookup is linear; local relinking is constant time only when the position is known.
- Node objects add allocation, header, reference, and locality costs in Java.
- Sentinels remove special cases when the head or tail changes.
- Reversal maintains a reversed prefix and untouched suffix.
- Preserve `next` before rewriting the current link.
- Recursive reversal consumes $O(n)$ stack space.
- Fast/slow pointers find middles and detect cycles.
- Cycle and intersection algorithms compare node identity, not value equality.
- A fixed pointer gap locates positions relative to the end.
- Stable merging can reuse nodes but transfers and mutates ownership.
- Linked-list merge sort is $O(n \log n)$ and relinks without an auxiliary element array.
- Pointer switching finds acyclic intersection in $O(m + n)$ time.
- Palindrome checking may require restoring the mutated second half.
- Doubly linked removal must update both neighbor directions.
- LRU combines a hash map with a doubly linked recency list.
- Composite map/list concurrency needs one synchronization strategy.
- Random-reference copying requires identity mapping unless temporary interleaving is allowed.
- Immutable lists can share tails safely.
- Test node identities, cycles, boundaries, mutation, and restoration explicitly.

Likely interview question: Why is linked-list insertion not simply $O(1)$?

Relinking at a known node is $O(1)$, but finding the insertion position is $O(n)$ when only the head and an index or search key are given. A complete answer separates navigation cost from mutation cost. It should also mention node allocation, cache locality, and ownership, because asymptotic pointer updates alone do not make a linked list faster than an array-backed list.

Likely interview question: How does Floyd's algorithm find the cycle entry?

The slow pointer advances one step and the fast pointer two. If they meet, a cycle exists. Reset one pointer to the head, keep the other at the meeting point, and advance both one step at a time; their next meeting is the entry. The distance relationship created by the first meeting makes the head-to-entry distance congruent to the meeting-to-entry distance around the cycle.

Likely interview question: How would you make an LRU cache thread-safe?

The map and recency list form one composite invariant: every cached key maps to exactly one linked node, and every linked node is represented in the map. Protecting only the map is insufficient. A straightforward design uses one lock around lookup-plus-move, insertion, eviction, and removal. More scalable designs may segment state or use an approximate policy, but they must explicitly define atomicity and accept the changed eviction semantics.

Likely interview question: Why restore a list after a palindrome check?

Reversing half the list is efficient, but leaving it reversed violates a non-mutating contract and can break later users. Restore temporary mutation even when comparison fails. This tests ownership.